

Module1-design principles

SOLID Principles

What is SOLID?

SOLID is a set of **5 Object-Oriented Design (OOD) principles** introduced by **Robert C. Martin (Uncle Bob)**.

Why SOLID?

- Write clean and maintainable code.
 - Reduce code complexity.
 - Improve scalability.
 - Promote code reuse.
 - Make applications easier to test and extend.
-

1. Single Responsibility Principle (SRP)

Definition

A class should have **only one responsibility** and **one reason to change**.

Key Point

One Class = One Responsibility

Example

Bad

```
class Employee:
    def calculate_salary(self):
        pass

    def save_to_database(self):
        pass
```

```
def send_email(self):  
    pass
```

Problem:

- Employee class has multiple responsibilities.

correct one

```
class SalaryCalculator:  
    def calculate_salary(self):  
        print("Calculating salary")  
  
class EmployeeRepository:  
    def save(self):  
        print("Saving employee")  
  
class EmailService:  
    def send_email(self):  
        print("Sending email")
```

Benefits:

- Easy to maintain
- Easy to test
- High cohesion

Interview Answer:

| Every class should do only one job.

2. Open-Closed Principle (OCP)

Definition

Software entities should be **open for extension** but **closed for modification**.

Key Point

Add new functionality without modifying existing code.

Example

Bad

```
class Payment:

    def pay(self, method):
        if method == "card":
            print("Card Payment")
        elif method == "upi":
            print("UPI Payment")
```

Problem:

Adding every new payment method requires modifying this class.

Good

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self):
        pass

class CardPayment(Payment):

    def pay(self):
        print("Card Payment")

class UpiPayment(Payment):

    def pay(self):
        print("UPI Payment")
```

To add Net Banking:

```
class NetBankingPayment(Payment):  
  
    def pay(self):  
        print("Net Banking Payment")
```

Benefits:

- Easy to extend
- Existing code remains unchanged

Interview Answer:

| Extend existing functionality instead of modifying existing code.

3. Liskov Substitution Principle (LSP)

Definition

A child class should be able to replace its parent class without changing program behavior.

Key Point

Child class should behave like Parent class.

Example

Bad

```
class Bird:  
  
    def fly(self):  
        print("Flying")  
  
class Penguin(Bird):
```

```
def fly(self):  
    raise Exception("Penguins can't fly")
```

Problem:

Penguin cannot substitute Bird.

Good

```
class Bird:  
    pass  
  
class FlyingBird(Bird):  
  
    def fly(self):  
        print("Flying")  
  
class Sparrow(FlyingBird):  
    pass  
  
class Penguin(Bird):  
    pass
```

Benefits:

- Proper inheritance
- No unexpected behavior

Interview Answer:

| A subclass should work anywhere its parent class is expected.

4. Interface Segregation Principle (ISP)

Definition

A class should **not be forced to implement methods it doesn't use.**

Key Point

Many small interfaces are better than one large interface.

Example

Bad

```
class Worker:

    def work(self):
        pass

    def eat(self):
        pass

class Robot(Worker):

    def work(self):
        print("Working")

    def eat(self):
        raise Exception("Robot cannot eat")
```

Problem:

Robot is forced to implement eat().

Good

```
class Workable:

    def work(self):
        pass

class Eatable:

    def eat(self):
        pass
```

```
class Human(Workable, Eatable):  
    pass
```

```
class Robot(Workable):  
    pass
```

Benefits:

- Cleaner interfaces
- Better flexibility

5. Dependency Inversion Principle (DIP)

Definition

High-level modules should depend on **abstractions**, not concrete implementations.

Key Point

Depend on Interfaces, not Implementations.

Example

Bad

```
class MySQL:  
  
    def connect(self):  
        print("Connected to MySQL")  
  
class App:  
  
    def __init__(self):  
        self.db = MySQL()
```

Problem:

App is tightly coupled with MySQL.

Good

```
from abc import ABC, abstractmethod

class Database(ABC):

    @abstractmethod
    def connect(self):
        pass

class MySQL(Database):

    def connect(self):
        print("Connected to MySQL")

class PostgreSQL(Database):

    def connect(self):
        print("Connected to PostgreSQL")

class App:

    def __init__(self, db):
        self.db = db

app = App(MySQL())
app.db.connect()
```

Benefits:

- Loose coupling
- Easy testing

- Easy to replace implementations

Interview Answer:

| Depend on abstractions instead of concrete classes.

Simply

Principle	Remember
SRP	One class → One responsibility
OCP	Extend, don't modify
LSP	Child should replace Parent
ISP	Don't force unused methods
DIP	Depend on abstractions

- ✓ SRP → One class should do one job.
- ✓ OCP → Extend functionality without modifying existing code.
- ✓ LSP → Child class should correctly replace parent class.
- ✓ ISP → Create small, specific interfaces.
- ✓ DIP → Depend on interfaces (abstractions), not concrete implementations.